

上海電力大學

JavaEE 课程报告



题 目： 易宿酒店预订管理系统的设计与实现

学 号：

姓 名：

院 系：

专业年级：

目录

第一章 引言.....	1
1.1 项目背景.....	1
1.2 项目意义.....	1
第二章 需求分析.....	3
2.1 功能需求.....	3
2.2 技术要求.....	4
第三章 系统设计.....	5
3.1 系统架构.....	5
3.2 模块划分.....	6
3.3 数据库设计.....	7
3.4 接口设计.....	9

3.5 技术选型.....	11
第四章 关键功能实现.....	13
4.1 登录注册与密码加密功能.....	13
4.2 酒店组合查询功能.....	19
4.3 客房预订与退订功能.....	20
4.4 后台管理与批量删除功能.....	23
4.5 统计报表功能.....	24
第五章 系统总结.....	27
5.1 系统已实现的功能.....	27
5.2 心得体会.....	28

第一章 引言

1.1 项目背景

随着在线旅游和酒店预订平台的发展，用户出行前通常需要提前检索酒店、比较房型价格、登记入住人并完成预订。传统线下电话订房方式存在信息不透明、库存更新不及时、订单管理效率低等问题。为了模拟真实酒店预订平台的核心业务流程，本项目设计并实现一个“易宿”酒店预订管理系统。

本系统采用单体 Spring Boot 后端、Vue3 前端和 MySQL 数据库，前后端通过 REST 接口进行交互。项目围绕用户注册登录、酒店检索、客房预订、订单管理、评价、后台维护和统计报表等功能展开，重点体现 JavaEE 课程中三层架构、增删改查、多条件查询、表单校验、角色菜单、密码加密和防 SQL 注入等要求。

1.2 项目意义

本项目的意义在于：

(1) 提供一个完整的酒店预订业务示例，覆盖用户浏览酒店、选择房型、登记入住人、提交订单、退订和评价等流程。

(2) 通过后台管理功能，管理员能够维护酒店和房型信息，处理订单状态流转，并查看用户与统计报表。

(3) 通过单体 Spring Boot 和 Vue 前后端分离结构，训练 Controller-Service-Mapper 三层架构设计与接口联调能力。

(4) 通过 BCrypt 密码加密、MyBatis 参数绑定、数据库约束和后端校验，提高系统的安全性和健壮性。

(5) 项目实现后可以作为课程答辩材料，清晰说明从需求、数据库、后端接口到前端页面的完整设计过程。

整体来看，本项目功能范围控制在简单档要求内，避免引入 Nacos、OpenFeign、Spring Security 和 JWT 等中等档技术。

第二章 需求分析

2.1 功能需求

(1) 用户管理：提供用户注册、登录、个人信息维护、积分和会员等级管理。

用户密码使用 BCrypt 加密后保存，订单完成后按照实付金额增加积分。

(2) 入住人管理：用户可以维护常用入住人信息，包括姓名、身份证号、性别、出生日期、职业、受教育程度和收入状况。

(3) 酒店检索：用户可以按照城市、区域、酒店名称、价格区间等条件进行组合查询，未登录用户也可以浏览酒店列表和评价。

(4) 客房预订：登录用户可以选择酒店房型、入住和离店日期、入住人并提交订单；系统需要校验日期、房型容量和库存。

(5) 订单管理：用户可以按状态查看自己的订单，查看订单详情，未入住订单可以退订，已完成订单可以评价一次。

(6) 后台管理：管理员可以维护酒店和房型信息，流转订单状态，查看用户列表和统计报表，房型维护支持表格多选批量删除。

2.2 技术要求

- (1) 编程语言：后端使用 Java 17，前端使用 JavaScript。
- (2) 后端框架：使用单体 Spring Boot，不使用 Nacos、OpenFeign、Spring Security、JWT 等中等档功能。
- (3) 前端框架：使用 Vue3、Vite 和 Element Plus，所有接口通过 Axios 请求后端。
- (4) 数据库：使用 MySQL 8，包含表、主外键、默认值、非空约束、CHECK 约束、索引、视图、触发器和存储过程。

第三章 系统设计

3.1 系统架构

本系统采用前后端分离的单体三层架构设计，包括：

- (1) 表示层：浏览器端运行 Vue3 单页应用，负责页面展示、表单校验、菜单切换和用户交互。
- (2) 业务逻辑层：Spring Boot 后端按 Controller、Service、Mapper 分层组织，Controller 接收请求，Service 处理业务规则，Mapper 访问数据库。
- (3) 数据访问层：MySQL 数据库负责存储用户、酒店、房型、订单、入住人、评价和统计数据。

图1 系统三层架构图

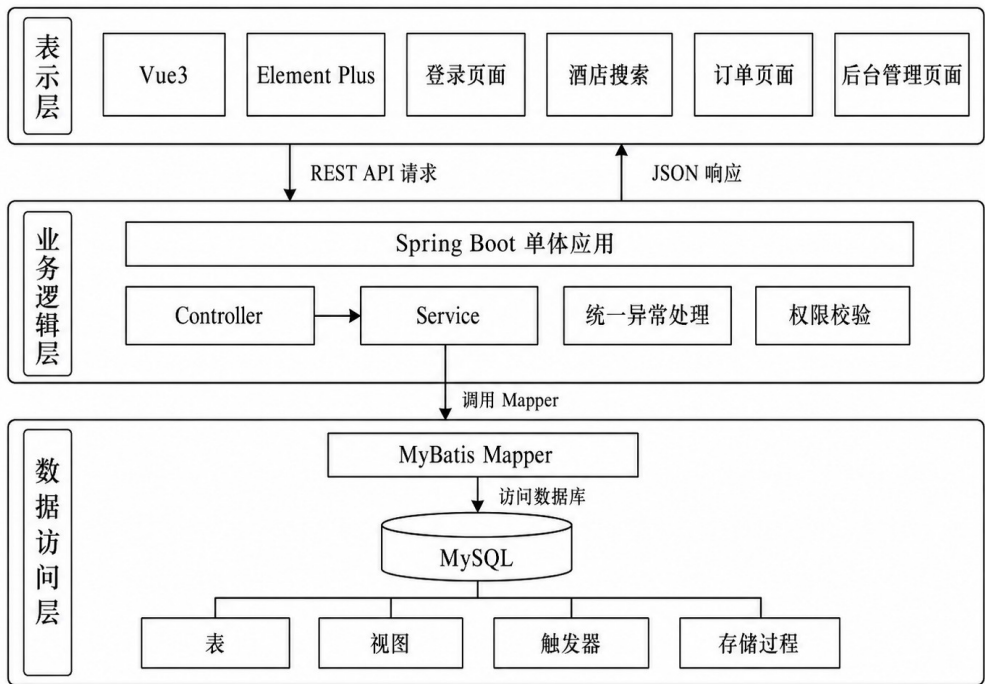


图 1 系统三层架构图

该架构把页面交互、业务规则和数据库访问分开，Controller 处理接口入口，Service 处理注册、预订、退订和统计业务，Mapper 负责 SQL 映射与参数绑定。

3.2 模块划分

系统主要分为以下模块：

- (1) 用户管理模块：负责用户注册、登录、个人信息维护、积分等级和常用入住人管理。
- (2) 酒店管理模块：提供酒店多条件查询、城市区域数据、酒店新增修改删除

和房型维护。

(3) 订单管理模块：提供创建订单、我的订单、订单详情、退订和管理员订单状态流转。

(4) 评价管理模块：用户完成订单后可以提交评价，酒店详情页可以展示评价列表。

(5) 统计报表模块：管理员可以查看预订统计、入住人画像统计和入住率统计。

(6) 权限菜单模块：普通用户进入首页、订单和个人中心；管理员额外进入后台二级菜单。

(7) 公共模块：包含统一返回 Result、全局异常处理、登录过滤和请求身份传递。

3.3 数据库设计

数据库设计包括以下主要表结构：

1.ER 图

图2 核心业务 ER 关系图

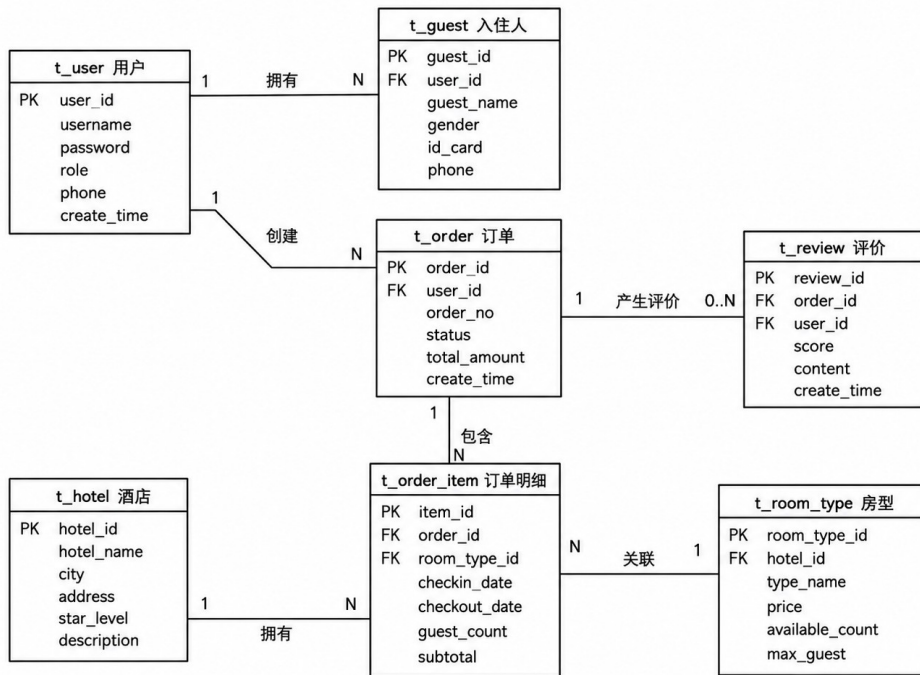


图 2 核心业务 ER 关系图

关系图中，用户与订单、入住人是一对多关系，酒店与房型是一对多关系，订单通过订单明细关联具体房型，便于库存扣减、订单统计和入住人画像统计。

2.表设计

(1) 用户表：存储用户账号、密码密文、手机号、身份证号、积分、会员等级和角色。

表 1 t_user 用户表结构表

字段	名称	字段类型	约束
user_id	用户编号	int	primary key

username	用户名	varchar(20)	not null, unique
password	密码密文	varchar(64)	not null
phone	手机号	char(11)	not null
role	角色	varchar(10)	用户/管理员

(2) 酒店表：存储酒店名称、城市、区域、地址、星级和联系电话等基础信息。

表 2 t_hotel 酒店表结构表

字段	名称	字段类型	约束
hotel_id	酒店编号	int	primary key
hotel_name	酒店名称	varchar(50)	not null
city	城市	varchar(20)	not null

(3) 房型表：存储酒店房型名称、价格、可住人数、总数量和可订数量。

表 3 t_room_type 房型表结构表

字段	名称	字段类型	约束
room_type_id	房型编号	int	primary key

hotel_id	酒店编号	int	foreign key
price	价格	decimal(10,2)	default 0
available_count	可订数量	int	check >= 0

(4) 订单表：存储订单编号、用户、酒店、入住离店日期、价格和订单状态。

表 4 t_order 订单表结构表

字段	名称	字段类型	约束
order_id	订单编号	int	primary key
order_no	订单号	varchar(32)	not null
user_id	用户编号	int	foreign key
hotel_id	酒店编号	int	foreign key
check_in_date	入住日期	date	not null
check_out_date	离店日期	date	大于入住日期
pay_price	实付价格	decimal(10,2)	default 0
status	订单状态	varchar(10)	已预订/已入住等

3.4 接口设计

系统提供以下主要 API 接口：

图4 前后端接口调用流程图

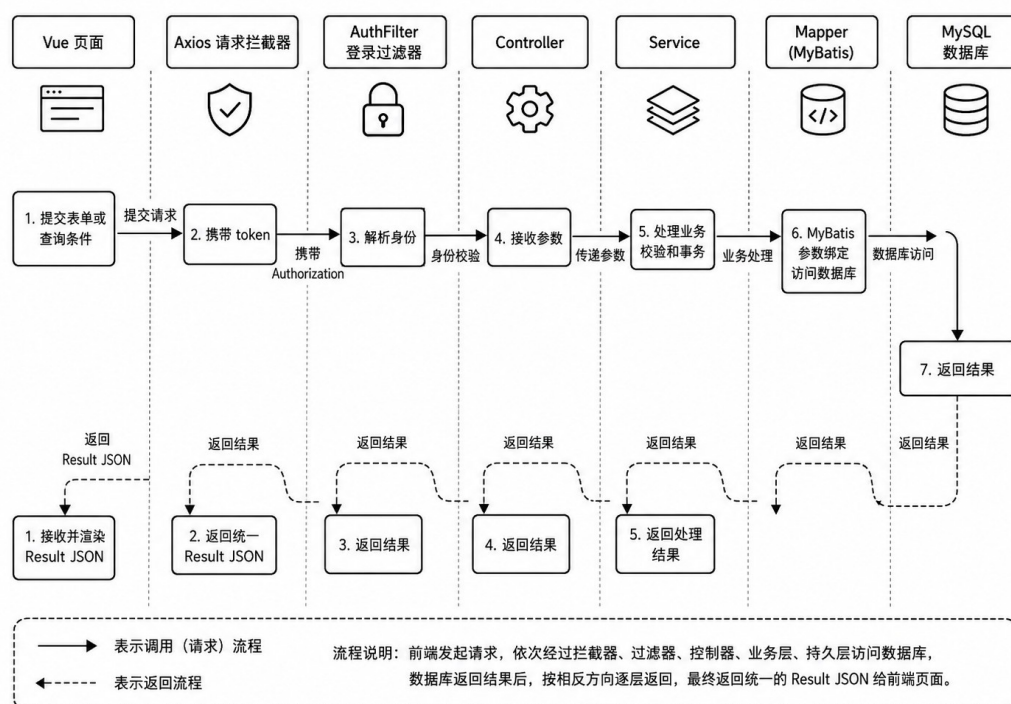


图 4 前后端接口调用流程图

接口调用统一经过 Axios、过滤器、Controller、Service 和 Mapper。前端

关注页面交互，后端关注权限、业务规则和数据库访问，层次边界清晰。

(1) 用户登录 API: /api/user/login, 校验用户名与 BCrypt 密码，登录成功
功后返回 token 和用户信息。

(2) 用户注册 API: /api/user/register, 校验表单后写入用户表，密码加密
后保存。

(3) 个人信息 API: /api/user/me, 查询或修改当前登录用户信息。

(4) 入住人 API: /api/guest/list、/api/guest, 维护用户常用入住人信息。

(5) 酒店查询 API: /api/hotel/search, 支持城市、区域、酒店名称和价格范围组合查询。

(6) 房型查询 API: /api/hotel/{hotelId}/rooms, 查询指定酒店可预订房型。

(7) 后台酒店 API: /api/hotel, 管理员新增、修改或删除酒店信息。

(8) 后台房型 API: /api/room、/api/room/batch, 管理员维护房型并支持多选批量删除。

(9) 订单 API: /api/order, 完成订单创建、我的订单查询和退订流程。

(10) 评价 API: /api/review, 已完成订单提交评价, 酒店详情页展示评价。

(11) 统计 API: /api/stats/booking, 供管理员查看预订、入住人和入住率统计报表。

3.5 技术选型

(1) Java: 作为后端开发语言, 配合 Spring Boot 实现 REST 接口和业务逻辑。

(2) Spring Boot: 作为单体后端框架, 启动和部署简单, 符合简单档课程要求。

(3) Vue3: 作为前端框架, 配合 Vite 和 Element Plus 实现前后端分离页

面。

(4) MySQL：作为数据库系统，支持约束、索引、视图、触发器和存储过程。

(5) BCrypt 和 MyBatis：BCrypt 用于密码加密，MyBatis 参数绑定用于

避免 SQL 拼接注入。

第四章 关键功能实现

4.1 登录注册与密码加密功能

(1) 技术实现

- ✓ 前端登录和注册页面使用 Vue3 与 Element Plus 表单组件，包含用户名、
密码、手机号、身份证号等输入项，并配置必填、长度和格式校验规则。
- ✓ 后端 UserService 对注册参数再次校验，使用 BCryptPasswordEncoder 对
密码加密后写入 t_user 表，避免明文密码存储。
- ✓ 登录时后端读取用户密码密文，通过 matches 方法校验用户输入密码，成功
后返回 Base64 形式 token 和用户角色。

图 3 登录认证与角色菜单流程图

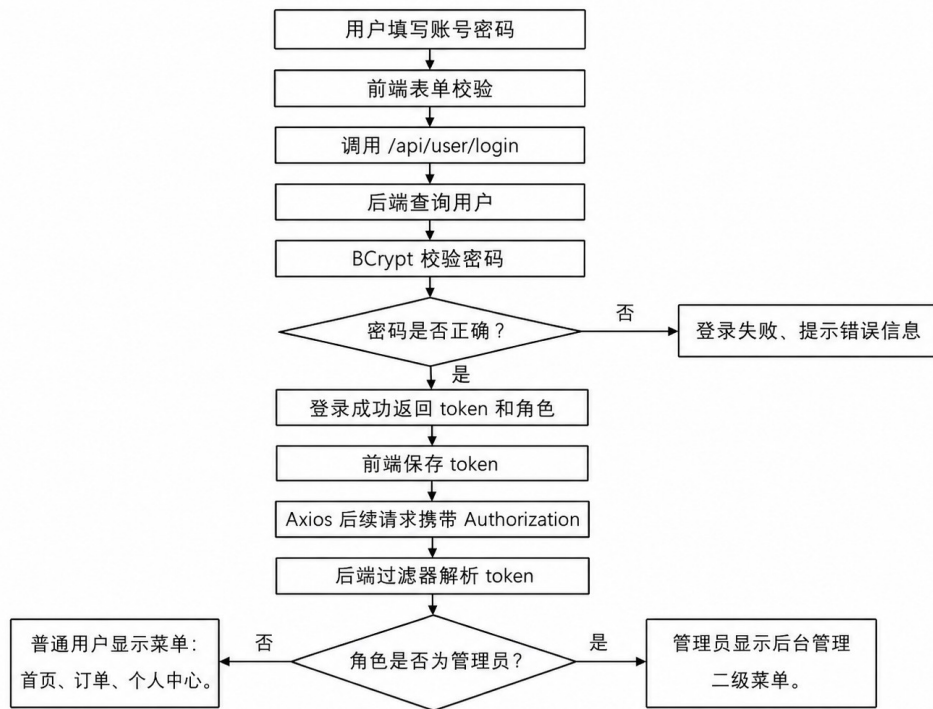


图 3 登录认证与角色菜单流程图

登录流程中，前端先做表单校验，后端再读取用户密码密文并使用 BCrypt 校验。登录成功后返回包含用户编号和角色的 token，后续请求由过滤器解析身份，页面根据角色展示普通用户菜单或管理员二级菜单。

代码 13 前端表单校验规则示例

```

const registerRules = {
  username: [
    { required: true, message: '请输入用户名', trigger: 'blur' },
    { min: 3, max: 20, message: '用户名长度为 3 到 20 位', trigger: 'blur' },
  ],
  password: [
    { required: true, message: '请输入密码', trigger: 'blur' },
    { min: 6, max: 20, message: '密码长度为 6 到 20 位', trigger: 'blur' },
  ],
  gender: [{ required: true, message: '请选择性别', trigger: 'change' }],
  education: [{ required: true, message: '请选择受教育程度', trigger:
'change' }],

```

```

    hobbies: [{ type: 'array', required: true, message: '请选择偏好标签',
trigger: 'change' }],
};

```

注册和入住人表单同时包含文本框、下拉框、单选和复选框。前端使用 Element Plus rules 做即时校验，后端 Service 再进行非空、长度、格式和枚举值校验，防止绕过前端直接调用接口。

代码 4 前端 Axios 请求拦截与异常处理

```

const request = axios.create({ baseURL: '/api', timeout: 10000 });

request.interceptors.request.use((config) => {
  if (auth.token) {
    config.headers.Authorization = auth.token;
  }
  return config;
});

request.interceptors.response.use(
  (resp) => {
    const { code, msg, data } = resp.data;
    if (code !== 0) return Promise.reject(new Error(msg));
    return data;
  },
  (error) => {
    if (error.response?.status === 401) {
      logout();
      window.location.hash = '#/login';
    }
    return Promise.reject(error);
  },
);

```

前端统一请求实例负责自动携带 token 和处理后端统一 Result，减少每个页面重复编写请求和错误处理代码。

(2) 前端界面

登录成功后，普通用户进入首页、订单和个人中心菜单；管理员进入后台管理菜单。

代码说明：

1. 前端在登录提交时调用 login API，将用户名和密码传给后端，并把返回的 token 保存到 localStorage。
2. Axios 请求拦截器会在后续请求中自动携带 Authorization 请求头，后端过滤器解析 token 后注入用户编号和角色。
3. 路由守卫根据用户角色控制菜单访问，管理员可以访问 /admin 下的二级菜单，普通用户不能进入后台页面。
4. 该功能同时体现了两种身份登录、不同菜单展示、前后端校验和密码加密等要求。

(3) 后端接口

主要接口包括 `/api/user/register`、`/api/user/login`、`/api/user/me` 和 `/api/user/list`。

代码说明：

代码 1 用户注册与登录密码加密核心代码

```
public void register(User user) {
    Long count = userMapper.selectCount(
        new LambdaQueryWrapper<User>().eq(User::getUsername,
            user.getUsername()));
    Assert.isTrue(count == 0, "用户名已存在");
    user.setPassword(PASSWORD_ENCODER.encode(user.getPassword()));
    userMapper.insert(user);
}

public Map<String, Object> login(String username, String password) {
    User user = userMapper.selectOne(
        new LambdaQueryWrapper<User>().eq(User::getUsername, username));
    Assert.isTrue(user != null &&
        PASSWORD_ENCODER.matches(password, user.getPassword()), "用户名或密码错误");
    String token = Base64.getEncoder().encodeToString(
        (user.getUserId() + ":" +
            user.getRole()).getBytes(StandardCharsets.UTF_8));
    return Map.of("token", token, "user", user);
}
```

以上代码体现了后端校验、用户名唯一性检查和 BCrypt 密码加密；登录时通过 matches 方法比较明文输入和数据库密文。

1. register 接口负责接收用户注册信息，调用 UserService 完成格式校验、用户名唯一性校验和 BCrypt 加密。
2. login 接口负责读取数据库用户信息并校验密码，校验成功后将 userId 和 role 编码为 token 返回前端。
3. me 接口用于登录用户查看和修改个人资料，用户列表接口只允许管理

员访问。

4. 后端通过 X-User-Role 请求头判断管理员权限，防止普通用户越权访问后台接口。

问后台接口。

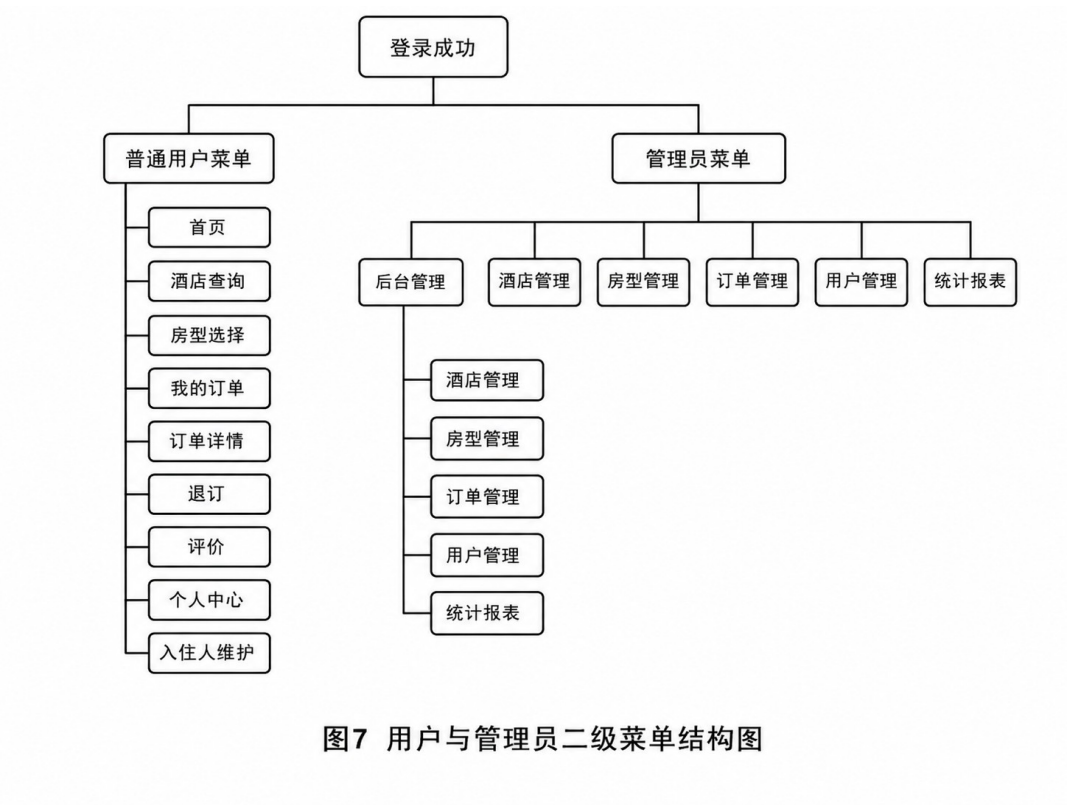


图7 用户与管理员二级菜单结构图

图 7 用户与管理员二级菜单结构图

代码 8 前端路由守卫与角色菜单控制

```
router.beforeEach((to, from, next) => {
  const token = localStorage.getItem('token');
  const role = localStorage.getItem('role');
  if (to.meta.requiresLogin && !token) {
    next('/login');
    return;
  }
  if (to.meta.adminOnly && role !== 'ADMIN') {
    next('/');
    return;
  }
})
```

```

        next();
    });

    const menus = computed(() =>
        auth.role === 'ADMIN' ? adminMenus : userMenus
    );

```

前端菜单分为普通用户菜单和管理员菜单。普通用户可以进入首页、酒店查询、我的订单和个人中心；管理员可以进入后台管理，并在后台下继续展开酒店管理、房型管理、订单管理和统计报表等二级菜单。

代码 5 登录过滤器解析 token

```

protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain chain) throws IOException,
ServletException {
    if (isWhitelisted(request)) {
        chain.doFilter(stripIdentityHeaders(request, null, null), response);
        return;
    }
    String token = request.getHeader("Authorization");
    try {
        String[] parts = new String(Base64.getDecoder().decode(token),
                                    StandardCharsets.UTF_8).split(":");
        String role = "管理员".equals(parts[1]) ? "ADMIN" : "USER";
        chain.doFilter(stripIdentityHeaders(request, parts[0], role),
response);
    } catch (Exception e) {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
    }
}

```

过滤器把 token 中的用户编号和角色转换为后端可信请求头，后台接口再根据角色判断管理员权限。

(3) 调用数据库

数据库访问通过 UserMapper 完成，查询条件使用 MyBatis 参数

绑定，不拼接 SQL 字符串。

(4) 运行结果

运行结果为：用户可以注册并登录，管理员登录后进入后台管理，普通

用户只能访问普通菜单。

4.2 酒店组合查询功能

酒店搜索页面提供城市、区域、酒店名称、价格范围等查询条件。前端将条件

作为请求参数提交到 /api/hotel/search；后端 HotelMapper.xml 使用动态

SQL，根据参数是否为空决定是否加入查询条件，并使用 #{参数名} 绑定变量

防 SQL 注入。

代码 2 酒店多条件组合查询动态 SQL

```
<select id="search" resultType="com.hotel.hotel.entity.Hotel">
    SELECT h.*, MIN(rt.price) AS min_price
    FROM t_hotel h JOIN t_room_type rt ON rt.hotel_id = h.hotel_id
    <where>
        <if test="city != null and city != ''">AND h.city = #{city}</if>
        <if test="district != null and district != ''">AND h.district =
#{district}</if>
        <if test="name != null and name != ''">AND h.hotel_name LIKE
CONCAT('%', #{name}, '%')</if>
        <if test="priceMin != null">AND rt.price >= #{priceMin}</if>
        <if test="priceMax != null">AND rt.price <= #{priceMax}</if>
```

```

        </where>
        GROUP BY h.hotel_id
    </select>

```

该 SQL 使用 MyBatis 动态标签按需加入查询条件，所有外部输入均通过 # {参数名} 绑定，避免字符串拼接 SQL。

4.3 客房预订与退订功能

用户选择房型、入住日期、离店日期和入住人后提交订单。OrderService 校验日期、房型容量和库存，创建订单及订单明细。数据库触发器在订单明细写入时扣减库存，退订时恢复库存。

图5 客房预订业务流程图

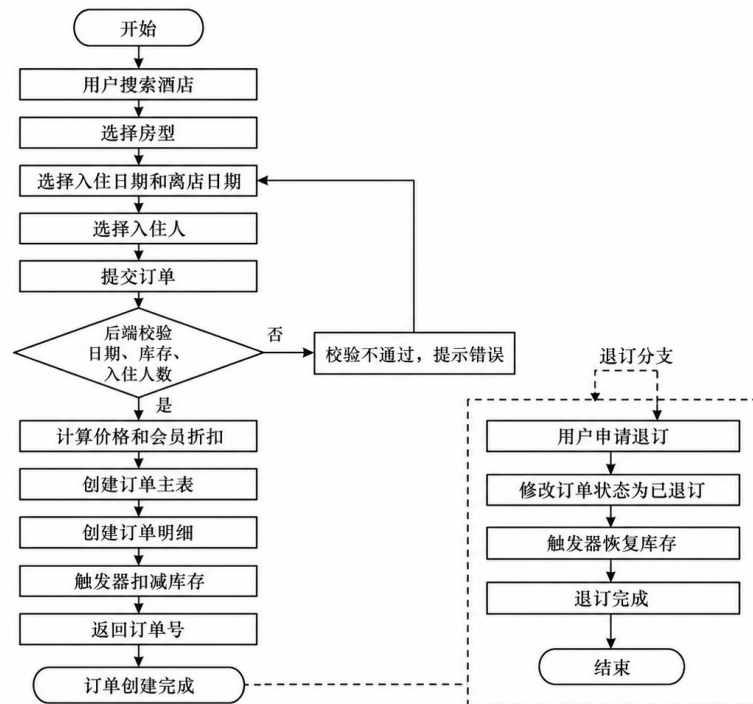


图 5 客房预订业务流程图

预订流程从前端选择酒店房型开始，后端统一校验入住日期、入住人数、房型

库存和会员折扣，最后写入订单主表与订单明细。

代码 6 订单创建核心事务代码

```
@Transactional
public String create(Integer userId, CreateOrderDTO dto) {
    LocalDate in = dto.getCheckInDate();
    LocalDate out = dto.getCheckOutDate();
    Assert.isTrue(out.isAfter(in), "离店日期必须晚于入住日期");
    Assert.notEmpty(dto.getRooms(), "请至少选择一间客房");

    Map<Integer, Integer> needByType = new HashMap<>();
    for (CreateOrderDTO.RoomReq room : dto.getRooms()) {
        RoomInfo info = orderMapper.selectRoomInfo(room.getRoomTypeId());
        Assert.isTrue(room.getGuestIds().size() <= info.getCapacity(), "入住人数超限");
        needByType.merge(room.getRoomTypeId(), 1, Integer::sum);
    }
    needByType.forEach((typeId, need) ->
        Assert.isTrue(need <=
            orderMapper.selectRoomInfo(typeId).getAvailableCount(), "库存不足"));
    orderMapper.insert(order);
    orderItemMapper.insert(item);
    return order.getOrderNo();
}
```

该方法使用事务保证订单主表、订单明细和入住人关系同时成功或同时回滚。

库存数量由数据库触发器扣减，业务层重点完成日期、容量和库存校验。

代码 12 库存维护触发器 SQL

```
CREATE TRIGGER trg_order_item_insert
AFTER INSERT ON t_order_item
FOR EACH ROW
BEGIN
    UPDATE t_room_type
    SET available_count = available_count - 1
    WHERE room_type_id = NEW.room_type_id
    AND available_count > 0;
END;
```

```

CREATE TRIGGER trg_order_cancel
AFTER UPDATE ON t_order
FOR EACH ROW
BEGIN
    IF NEW.status = '已退订' AND OLD.status <> '已退订' THEN
        UPDATE t_room_type rt
        JOIN t_order_item oi ON oi.room_type_id = rt.room_type_id
        SET rt.available_count = rt.available_count + 1
        WHERE oi.order_id = NEW.order_id;
    END IF;
END;

```

触发器把库存维护放在数据库侧执行，能减少业务代码遗漏库存更新的风险。

订单创建只负责写入订单明细，订单退订只负责修改订单状态，库存扣减和恢

复由触发器根据数据变化自动完成。

图6 库存触发器维护流程图

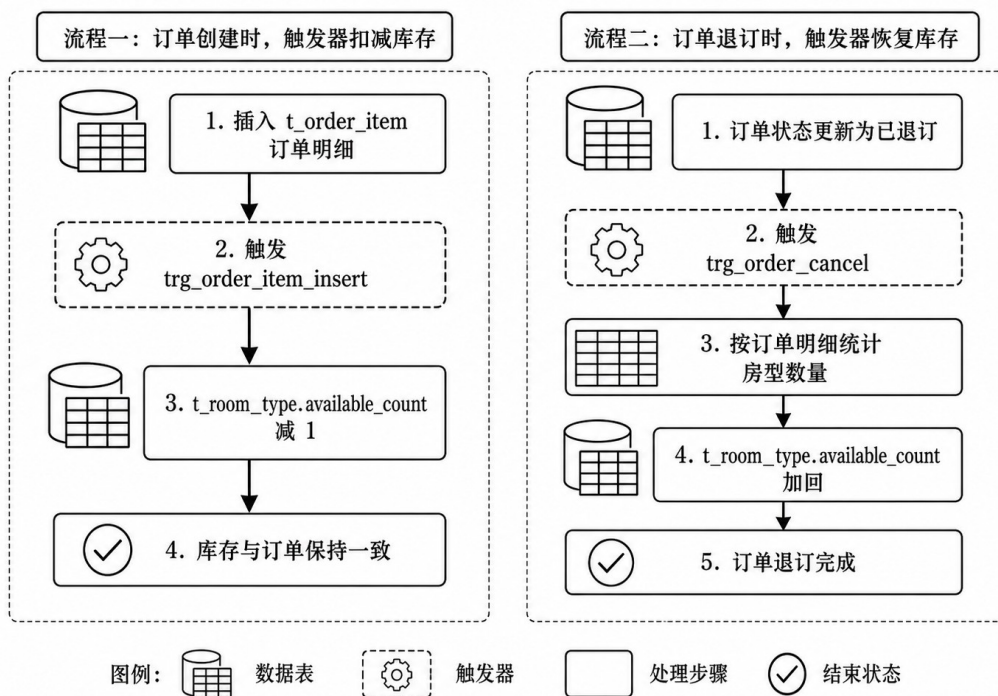


图 6 库存触发器维护流程图

库存不直接由前端修改，而是在订单明细插入和订单退订时由数据库触发器维护，避免并发操作下出现订单和库存不一致。

4.4 后台管理与批量删除功能

管理员后台包含酒店与房型、订单管理、用户管理和统计报表等二级菜单。房型维护页面使用表格多选列实现批量删除，前端调用 `DELETE /api/room/batch`，后端通过 `deleteBatchIds` 批量删除。

代码 9 Excel 批量导入房型核心代码

```
@PostMapping("/room/import")
public Result<Integer> importRooms(@RequestParam("file") MultipartFile file,
                                   @RequestHeader("X-User-Role") String role) {
    HotelController.checkAdmin(role);
    List<RoomType> rows = excelRoomReader.read(file);
    hotelService.importRoomTypes(rows);
    return Result.ok(rows.size());
}

@Transactional
public void importRoomTypes(List<RoomType> rows) {
    Assert.notEmpty(rows, "导入数据不能为空");
    for (RoomType room : rows) {
        Assert.notNull(room.getHotelId(), "酒店编号不能为空");
        Assert.hasText(room.getRoomName(), "房型名称不能为空");
        Assert.isTrue(room.getPrice().compareTo(BigDecimal.ZERO) >= 0, "价格不能为负数");
        roomTypeMapper.insert(room);
    }
}
```

批量新增采用 Excel 导入方式实现，管理员上传表格后，后端逐行校验酒店编

号、房型名称、价格和库存数量。只要其中一行不符合规则，事务就会回滚，

避免出现部分导入成功、部分失败导致的数据不一致。

批量删除与批量导入共同覆盖了课程要求中的“支持批量新增或删除”。前端批

量删除适合后台表格操作，Excel 导入适合一次性录入多个房型，两者都放在

管理员菜单下，并且都要先进行角色校验。

代码 3 房型批量删除接口

```
@DeleteMapping("/batch")
public Result<Void> deleteBatch(@RequestHeader("X-User-Role") String role,
                                @RequestBody List<Integer> ids) {
    HotelController.checkAdmin(role);
    hotelService.deleteRoomTypes(ids);
    return Result.ok();
}
```

前端房型表格提供多选列，管理员选择多条记录后一次提交编号列表；后端先

校验管理员角色，再调用 Service 执行批量删除。

4.5 统计报表功能

系统提供预订统计、入住人画像统计和入住率统计。统计数据由 MySQL 存储

过程计算，前端 AdminStats 页面以表格方式展示汇总和明细。

代码 7 统计存储过程调用 Mapper

```
<select id="bookingStats" statementType="CALLABLE"
resultType="java.util.Map">
    {CALL sp_booking_stats(
        #{start, mode=IN, jdbcType=DATE},
```

```

        #{end, mode=IN, jdbcType=DATE},
        #{group, mode=IN, jdbcType=VARCHAR}
    })
</select>

<select id="occupancyRate" statementType="CALLABLE"
resultType="java.util.Map">
    {CALL sp_occupancy_rate(
        #{start, mode=IN, jdbcType=DATE},
        #{end, mode=IN, jdbcType=DATE}
    )}
</select>

```

统计模块把复杂汇总逻辑放到 MySQL 存储过程中，后端 Mapper 通过

CALLABLE 方式调用，前端只负责展示统计结果。

代码 10 统一异常处理与返回格式

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalArgumentException.class)
    public Result<Void> handleBadRequest(IllegalArgumentException e) {
        return Result.fail(e.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public Result<Void> handleServerError(Exception e) {
        log.error("system error", e);
        return Result.fail("系统繁忙，请稍后重试");
    }
}

```

统一异常处理用于保证前后端返回格式一致。业务层抛出的参数错误会转换为

明确提示，未知异常会写入日志并返回通用错误信息，避免将数据库异常或堆

栈信息直接暴露给前端。

代码 11 统计服务调用存储过程

```
public List<Map<String, Object>> bookingStats(LocalDate start, LocalDate end,
String group) {
    Assert.notNull(start, "开始日期不能为空");
    Assert.notNull(end, "结束日期不能为空");
    Assert.isTrue(!end.isBefore(start), "结束日期不能早于开始日期");
    return statsMapper.bookingStats(start, end, group);
}
```

统计接口仍保持 Controller-Service-Mapper 三层结构。Controller 只接收

参数，Service 负责日期范围校验，Mapper 负责调用 MySQL 存储过程。

这样答辩时可以清楚说明每一层的职责边界。

第五章 系统总结

5.1 系统已实现的功能

(1) 用户管理：完成用户注册、登录、个人信息维护、积分等级和常用入住人管理。

(2) 酒店检索：完成城市、区域、酒店名称、价格范围等多条件组合查询。

(3) 订单预订：完成房型选择、入住人登记、订单创建、我的订单、订单详情和退订。

(4) 后台管理：完成管理员酒店维护、房型维护、房型批量删除、订单状态流转和用户列表查看。

(5) 统计报表：完成预订统计、入住人统计和入住率统计，并在前端后台页面展示。

(6) 安全规范：完成 BCrypt 密码加密、MyBatis 参数绑定、防 SQL 注入、统一异常处理和角色权限控制。

5.2 心得体会

通过本次课程设计，我对前后端分离项目的完整开发流程有了更系统的认识。

从需求分析、数据库建模、后端三层架构、前端页面组织，到接口联调和构建

验证，每一步都需要保持数据结构、接口参数和页面交互的一致性。在后端实

现中，Controller 负责接收请求和权限入口判断，Service 负责业务规则和

事务控制，Mapper 负责数据库访问。这种分层方式让代码职责更加清晰，也

便于在答辩时说明每一层的作用。在数据库设计中，主外键、CHECK 约束、

视图、触发器和存储过程分别解决了完整性、明细复用、库存自动维护和统计

计算等问题。在安全和规范方面，BCrypt 密码加密和 MyBatis 参数绑定让

我认识到课程项目也应避免明文密码和 SQL 拼接这类基本风险。整体来看，

本项目基本覆盖了简单档要求，同时也为后续继续扩展为更复杂的系统打下了

基础。